An Anonymous Self-Stabilizing Algorithm For 1-Maximal Matching in Trees

Wayne Goddard, Stephen T. Hedetniemi and Zhengnan Shi

May 2005

Abstract

We present an anonymous self-stabilizing algorithm for finding a 1-maximal matching in trees (and rings of length not divisible by 3). We show that the algorithm converges in $O(n^4)$ moves under an arbitrary central daemon.

1 Introduction

Networks of computers are used to solve a wide variety of problems. Most services for a network involve maintaining a global predicate over the entire network by using local information at each participating node. To this end, several paradigms for distributed algorithms have been studied. Among these are the (fault-tolerant) self-stabilizing algorithms introduced by Dijkstra [2]. The self-stabilizing property is that a system can start in any configuration and yet is guaranteed to converge to a desired configuration in finite time and remain so thereafter [3, 4, 10]. In this paper we provide a self-stabilizing algorithm which produces an improved maximal matching.

[0]Second and third authors supported in part by NSF grant #0218495

1.1 Self-stabilization

In the self-stabilizing algorithmic paradigm, each node maintains and changes its own local variables based on the current values of its variables and those of its ***neighbors***. The values of the local variables at a node determine its ***local state***. The ***global state*** of a network is the union of all the ***local states***.

When a node changes its local state, it is said to make a ***move***. The algorithm is presented as a set of rules. Each rule is of the form "*if* $p(i)$ *then* $q(i)$" where i is a node, $p(i)$ is a boolean predicate and $q(i)$ a move. A node is said to be ***privileged*** if $p(i)$ is true. If a node becomes privileged, it may execute the corresponding move $q(i)$.

When no node is privileged, we say that the system is in a ***stable state***. For us, an algorithm is ***self-stabilizing*** if: from any initial global state it reaches a legitimate global stable state after a finite number of moves. (This is sometimes called a silent self-stabilizing algorithm.)

In this paper we assume a ***central daemon*** [2, 7], which at each time-step selects one of the privileged nodes to make a move: thus no two nodes move at the same time. We do not make use of IDs: the network is ***anonymous***. However, the ability of anonymous, self-stabilizing algorithms to solve problems is relatively weak, and thus our algorithm works only for limited networks. We assume ***coarse atomicity***: a node can read all its neighbors' variables and update its own variables in one move.

Anonymous self-stabilizing algorithms for various graph problems have been given in [5, 6, 7, 8, 9].

1.2 Our Result

We model a distributed network as a connected, undirected graph $G = (V, E)$. For a node $i \in V$, $N(i)$, its ***open neighborhood***, is the set of nodes to which i is adjacent. A ***matching*** is a subset M of edges such that each node in V is incident to at most one edge in M. A matching is

maximal if $\forall e \in E - M$ it holds that $M \cup \{e\}$ is not a matching.

An anonymous self-stabilizing algorithm for maximal matching in any connected network was presented by Hsu and Huang [7]. Though the algorithm uses pointers and appears to use IDs, a node only needs to know whether a neighbor points to (a) it, (b) nothing, or (c) another node. A maximal matching algorithm for a model with fewer assumptions (but using randomness to generate IDs) was presented by Chattopadhyay, Ligham and Seyffarth [1].

A maximal matching M is said to be ***1-maximal*** if it satisfies the following property: $\forall e \in M$, no matching can be constructed by removing e from M and adding two edges to $M - \{e\}$.

We present an anonymous, self-stabilizing algorithm for finding a 1-maximal matching. The algorithm produces a 1-maximal matching in any network if it stabilizes. Furthermore, when the underlying graph is a tree or a ring whose length is not divisible by 3, the algorithm is guaranteed to stabilize.

We show that the performance guarantee of a 1-maximal matching algorithm is better than that of a maximal matching algorithm. We also show that for rings of length a multiple of 3, no deterministic, anonymous, 1-maximal matching algorithm is possible.

2 Performance and Optimality

It is well known that a maximal matching has cardinality at least half that of a maximum matching. A better bound holds for 1-maximal matchings.

Lemma 2.1 *Any 1-maximal matching has at least two-thirds the cardinality of a maximum matching.*

PROOF. Let M be a 1-maximal matching and let N be a maximum matching. Consider the graph H induced by the edges of $M \cup N$. This graph has maximum degree (at most) two: every component C is a path or even cycle.

Assume the number of N-edges exceeds the number of M-edges in C. Then C is an odd-length path which alternates between M-edges and N-edges, and starts and finishes with an N-edge. If C has one edge, then that edge can be added to M thereby contradicting the maximality of M. If the path has three edges, then the middle edge can be removed from M and the other two edges added to M to form a bigger matching, a contradiction of the 1-maximality of M. So C has at least five edges.

It follows that in C the number of M-edges is at least $2/3$ the number of N-edges. Since this applies to every component of H, the theorem follows. QED

We show next that in a ring of length a multiple of 3, no anonymous 1-maximal matching algorithm is possible. (The maximum matching algorithm for bipartite graphs given in [1] is not anonymous—the claim in their introduction is incorrect.)

Lemma 2.2 *No anonymous self-stabilizing algorithm on a ring of order n a multiple of 3 can guarantee more than the smallest maximal matching (which has cardinality $n/3$), even under the central daemon.*

Proof. This follows by a standard symmetry argument. Color the nodes with three colors such that every third node has the same color. The central daemon will ensure that, for each color, every node of that color always has the same state. The daemon starts them this way, and if one node of a given color is privileged, then all nodes of that color are privileged, so the daemon can select all nodes of that color in succession. Thus, at stabilization, the solution is symmetric under rotation through three nodes, and hence is a maximal matching of cardinality $n/3$. QED

3 Description of Algorithm

We now present our algorithm. It builds on the maximal matching algorithm of Hsu and Huang [7]. In that algorithm, every node maintains a pointer. When the system stabilizes, the pointers define the matched edges. (Each matched node points to what we call its ***spouse***.)

To obtain a 1-maximal matching algorithm, we need to provide a mechanism for an ***two-for-one exchange*** where one matched edge is replaced by two. This is achieved by a ***hand-shaking*** mechanism: the matched nodes signal to each other the availability of alternative spouses, lock in a commitment from these alternatives, and then point to their new spouse.

3.1 The local variables

While the ideas underlying them are not that complex, the rules of our algorithm are complicated due to the possibility of bad initial data and the adversarial nature of the daemon. Hence, we present first a simplified version of the rules in informal language. Of necessity, certain concepts will not be defined precisely (or will be inaccurate) until we present the actual rules.

We start with an explanation of the local variables. In the algorithm, each node i maintains two variables: (i) a pointer, and (ii) a local variable x_i. The pointer records the neighbor to which i points (its intended spouse). We write $i \to j$ if i's pointer is set to j, and $i \to \emptyset$ if i's pointer is set to null.

The variable x_i records the current state of the node; this is used to indicate progress in an exchange. There are four states, called 0, 1, 2 and 3. Informally we can describe their purpose as follows:

- State 0 is a quiet state (and any unmatched node should be in this state).
- State 1 should mean that the (matched) node has an alternative "available" neighbor.

- State 2 should mean that the node's spouse also has an available neighbor. This state is a signal that beckons the available neighbor to point to (commit to) the node.
- State 3 should mean that the node has a committed neighbor. This is a signal to the spouse that the spouse can break the match if the spouse has likewise a committed neighbor.

3.2 The rules

We start with the intention of the rules, in informal language. We group the rules into three pairs.

The first pair of rules are designed for the *node that is not in a match*. These two rules echo the rules of the original maximal matching algorithm:

1. BackUp: if you are pointing to a node that does not want you to point to it, then set your pointer to null.
2. Point: if your pointer is null and there is a neighbor that does want you to point to it, then point to that neighbor.

The second pair of rules are designed for a *node that is matched* in order for it to implement the hand-shaking procedure. The state of the node is meant as a signal to the spouse: it indicates progress in informing nodes and obtaining commitments from nodes to do the two-for-one exchange.

3. StateIncrease: if you see the potential to expand the number of unmatched edges—because of your spouse's state or a suitable neighbor—then go to the next higher state.
4. Abort: if your state does not correctly represent the situation—because of changes by your neighbors or because your data was not initialized—then reset your state.

The final pair of rules are designed to enable the two-for-one exchange to occur: the two spouses each marry a committed neighbor.

5. BreakMatch: if your spouse has agreed to the exchange, and you have a suitable neighbor, then change to the neighbor (and reset your state).

6. Remarry: if your spouse has already changed, then change to a committed neighbor.

We now present the exact rules. We need some terminology.

Definition. *A node i is in a **match** with node j if $i \to j$ and $j \to i$. In this case, j is its **spouse**. This is a **proper match** if $|x_i - x_j| \leq 1$.*

For convenience we distinguish certain neighbors of a node i:

Definition. *A neighbor $l \in N(i)$ is **beckoning** to i if $i \nrightarrow l$ and $x_l = 2$.*
*A neighbor $k \in N(i)$ is **incoming** if $i \nrightarrow k$ and $k \to i$.*
*A neighbor $j \in N(i)$ is **open** if $i \nrightarrow j$ and $j \to \emptyset$.*

(The concept of available neighbor from the informal description corresponds to a neighbor that's incoming or open.)

The rules of the algorithm are shown in Figure 1.

(1) BackUp: **if** $i \rightarrow j \ \wedge \ j \nrightarrow i \ \wedge \ j \nrightarrow \emptyset \ \wedge \ x_j < 2$
then $x_i = 0 \ \wedge \ i \rightarrow \emptyset$

(2) Point: **if** $i \rightarrow \emptyset$
$\wedge \ \exists$ incoming, beckoning or open $k \in N(i)$
then $x_i = 0 \ \wedge \ i \rightarrow k$
(the selection order for k is incoming neighbors in increasing state, then beckoning neighbors, then open neighbors)

(3) StateIncrease: **if** $x_i < 3$
$\wedge \ i$ is in a proper match with j with $x_i \leq x_j$
$\wedge$ (*either* $\exists$ incoming $k \in N(i)$, *or* $x_i < 2$ and $\exists$ open $k \in N(i)$)
then $x_i{+}{+}$

(4) Abort: **if** $x_i > 0 \ \wedge$ not eligible for Rule Remarry $\wedge$
$\wedge$ (*either* i is not in a proper match, *or* $\nexists$ incoming nor open $k \in N(i)$, *or* $x_i = 3$ and $\nexists$ incoming $k \in N(i)$)
then set $x_i = 2$ if $x_i = 3$, i's spouse is in State 1 and i has an incoming neighbor; and set $x_i = 0$ otherwise.

(5) BreakMatch: **if** i is in a proper match with a spouse in State 3
$\wedge \ \exists$ incoming $k \in N(i)$ that is not in State 3
then $x_i = 0 \ \wedge \ i \rightarrow k$

(6) Remarry: **if** $x_i = 3 \ \wedge \ i \nrightarrow \emptyset \ \wedge \ i$ is not matched
$\wedge \ \exists$ incoming $k \in N(i)$
then $x_i = 0 \ \wedge \ i \rightarrow k$

Figure 1: The 1-Maximal Matching Algorithm (the rules for node i)

4 Correctness

In this section we show that every stable state has the desired global property. To show this correctness, we show that at convergence there is no node in state 2 or 3. Then we show that this means the pointers represent a 1-maximal matching.

For a value y of the state variable x, we use the notation $\mathcal{S}(y)$ to represent the set of nodes i for which $x_i = y$.

Lemma 4.1 *In a stable state produced by this algorithm, $\mathcal{S}(2) = \mathcal{S}(3) = \emptyset$.*

PROOF. Suppose upon stabilization there is a node $u \in \mathcal{S}(2)$. Because u is not privileged for Rule Abort, it must be in a proper match.

Let v be u's spouse: we know $x_v \geq 1$. If $x_v = 1$, then v is privileged for either Rule StateIncrease or Abort. So assume $x_v \geq 2$. Then the only way u is privileged for neither of those rules, is if it has an open neighbor, say z. However, then node z is privileged for Rule Point, a contradiction. Thus in a stable state, $\mathcal{S}(2)$ is empty.

Suppose there is a node $w \in \mathcal{S}(3)$. Since $\mathcal{S}(2) = \emptyset$, and w cannot execute Rule Abort, w is in a proper match with a spouse in State 3. The node w cannot have an incoming neighbor in State 3, since that neighbor would be privileged for Rule Abort. Thus w is privileged either for Rule Abort or for Rule BreakMatch, depending on whether it has an incoming neighbor or not. This is a contradiction. QED

Lemma 4.2 *In any network, if the algorithm stabilizes then it produces a 1-maximal matching given by nodes pointing to each other.*

PROOF. By Lemma 4.1, in a stable state the network contains nodes only in States 0 and 1. By Rule BackUp and Rule Point, if $i \rightarrow j$ then $j \rightarrow i$. Thus, a matched edge is given by spouses pointing to each other. The result is a matching.

Suppose the result is not maximal. Then there is a pair of adjacent nodes whose connecting edge can be added: but they are privileged for Rule Point.

Similarly, if the result is not 1-maximal, a pair of adjacent nodes must be privileged for Rule StateIncrease, a contradiction. QED

5 Convergence and Complexity Analysis

We need to prove that in some networks the algorithm always reaches a stable state after a finite number of moves. There are two parts to showing convergence.

The first part is to introduce the concept of a ***progress move***. We show that the number of progress moves is bounded. A useful concept here is the idea of a node being ***semi-matched***. The set of matched nodes does not monotonically increase—we need the ability to break matches and remarry. Nevertheless, we show that once a node becomes semi-matched, it remains semi-matched (and we later show that the node will eventually become matched in the special networks). The first part holds true in any network.

For the second part, we show that, for the special networks we are interested in, the daemon cannot cause the network to move indefinitely without a progress move occurring. The idea here is that if the process goes on forever, there must be nodes which are ***repeatedly starting the exchange process*** and then aborting it. Since there is no overall progress, this abortion is caused by an available neighbor committing itself to another exchange. That exchange in turn is aborted because of another neighbor committing elsewhere, and so on. This process can go on forever if there is a cycle back to the first exchange. However, we show that an infinite process is impossible in trees, and in rings whose length is not divisible by 3.

5.1 Semi-matched nodes and progress moves

We define the following concept.

Definition. *A node is* ***dirty*** *if it has not moved, otherwise it is* ***clean****.*

Lemma 5.1 *The number of matches never decreases.*

PROOF. The only move (Rule BreakMatch) that deletes an existing match replaces it by another match. QED

Definition. *A node is* ***semi-matched*** *if*
a) it points to null and is pointed to by a node that is not in state 3, or
b) it is in state 3 and is pointed to (but not matched).

Let T denote the set of nodes that are matched or semi-matched. The first lemma shows that one a node actively becomes semi-matched, then it remains so.

Lemma 5.2 *A node u in T remains in T except possibly when u is dirty and in State 3.*

PROOF. (1) Assume u is semi-matched. Then all incoming pointers are frozen (though such neighbors may execute Rule Abort). Thus the only way u could stop being semi-matched is if it itself moves. But the only pointer move that it can make is Rule Point or Remarray, so u becomes matched, and hence remains in T.

(2) Assume u is matched. A node that is matched remains matched, until one of the matched pair executes Rule BreakMatch. If u breaks the match, then it changes to match to another node and so remains in T. So assume that the spouse of u moves. It follows that u is in State 3, and by the hypothesis, u is clean. Since u has already moved, it has an incoming neighbor k, whose pointer is frozen; since u is in State 3 it thus remains semi-matched. QED

Definition. *A* ***progress move*** *is:*
Type 1. a move of a dirty node in State 3,
Type 2. a move that creates a new match, or
Type 3. a move that changes a null pointer to point to an open neighbor.

Lemma 5.3 *There are at most $O(n^2)$ progress moves.*

PROOF. Clearly Type 1 progress moves occur at most n times.

We claim that every Type 2 move increases the set T. For, consider a move by u that creates a match with v. We may assume u was already in T. There are two cases. If u was matched, then it must execute Rule BreakMatch, which means that v was not matched, not pointing to null, and not in State 3, and hence was not semi-matched. That it, v was not in T before.

So assume u was semi-matched before the move. If u is in State 3 but not matched, then it can only execute Rule Abort; so it must be that u points to null. Then v is not matched and does not point to null. If v were in State 3. But then by the predicate of Rule Point, u would not choose v, since it has a neighbor of lower state. So v is not in State 3, and in particular, is not semi-matched.

Thus every Type 2 move adds a node to T. Indeed, the above argument shows that every Type 2 move adds either a clean node or a node not in State 3 to T. By the above lemma, this can happen at most n times.

A Type 3 move freezes that node; the only move the open neighbor can ever make is to point to one of its incoming neighbors. That move will create a match. So the number of Type 3 moves is at most n times the number of new matches, which means it is at most $O(n^2)$. QED

5.2 Between progress moves in special networks

In many networks, however, the daemon can cause live-lock where no progress move occurs and yet the algorithm does not terminate. So in some special networks, which include trees and cycles of the appropriate length, we next argue that the adversarial daemon cannot go on forever without causing progress.

We therefore focus on the maximum number of moves that can be made between two progress moves. During such a period, the set of matched edges remains unchanged. Indeed, the nodes that are matched can execute only

Rule StateIncrease or Abort.

A node that is not matched cannot change its pointer to an incoming or open neighbor, since both are progress moves. We define a u-to-v ***flip*** as a move by node u that changes its pointer from null to point to a beckoning neighbor v. We define a ***flop*** as a node executing Rule BackUp. We define a ***flopflip*** as a flop followed by a flip.

Lemma 5.4 *Consider a period during which there is no progress move. Assume a node u flopflips to v three times. Then between the first flop and the third flip, some node x adjacent to v's spouse w flopflips to a node other than w.*

Proof. At the time of the first flop, node v is in state at most 1. If v is not matched, it cannot increase its state. So v has a spouse, call it w. At the time of the first flip, node v is in State 2 and hence at some stage w is in state at least 1.

At the time of the second flop, node v is again in state at most 1: it executed Rule Abort. At that moment, w was in State 0; thus w has also executed Rule Abort. At the time of the second flip, v is in State 2. Hence at some moment w was in state at least 1; at the moment that w increased state it had an open or incoming neighbor, call it x.

At the time of the third flop, node v is again in state at most 1: it executed Rule Abort. At that moment, w was in State 0; thus w has executed Rule Abort and thus x flipped to some other node. At the time of the third flip, v is in State 2. Hence at some moment w was in state at least 1; at the moment that w increased state, x was open or incoming neighbor and hence it had flopped. QED

Corollary 5.1 *Consider a period during which there is no progress move. If a node u flopflips to v k times, then one can identify a sequence of $k-2$ flopflips $f_1, \ldots, f_{k-2}$ as follows. The flopflip f_i is executed by some neighbor w of v, distinct from u, and points to some node other than v or u, and the flop part of f_i occurs between the ith flop by u and the $(i+2)$nd u-to-v flip.*

We define the ***cause*** of a flopflip as follows. We consider the first two u-to-v flopflips as ***self-caused***. For a later u-to-v flopflip, we assign the ***cause*** as another flopflip by some node x, as in the above lemma. Associated with each cause, other than self-causes, is a directed ***cause path*** from u to x.

Now, the flopflip by x in turn, has a cause. We can therefore follow the cause paths until we reach a self-caused flopflip: that we define as the ***ultimate cause***.

Lemma 5.5 *In trees and rings of length not a multiple of* 3*, there are at most* $O(n^2)$ *flopflip moves between successive progress moves.*

PROOF. Consider a tree. As we follow the sequence of cause paths, the paths cannot reverse themselves nor have repeated nodes, and so one is always moving away from the source flopflip. It follows that the same node can be the ultimate cause of a u-to-v flopflip at most twice. Indeed, all ultimate causes of a u-to-v flopflip, apart from the self-caused ones, are in the subtree of $T - uv$ containing v.

It follows that there are at most $O(n)$ flopflips by u. This implies that the total number of flopflips, between two progress moves, is $O(n^2)$.

Consider a ring. Again the sequence of cause paths is moving away. Since, the ring has length not a multiple of 3, the sequence cannot wrap around the ring, because the paths must alternate between an unmatched node and two matched nodes. Thus the conclusion follows as in the case for trees. QED

Lemma 5.6 *There are at most* $O(n^2)$ *moves between progress moves in trees and rings of length not a multiple of* 3*.*

PROOF. The number of states is a constant. From Lemma 5.5, the number of flopflips is at most $O(n^2)$. The corresponding state changes by matched nodes must be of the same order. QED

Theorem 1 *The 1-maximal matching algorithm converges in* $O(n^4)$ *moves in trees and rings of length not a multiple of* 3*.*

PROOF. This follows from Lemmas 5.3 and 5.6. QED

Acknowledgement

We would like to thank the referee whose careful reading and encouragement helped improve the paper.

References

[1] S. Chattopadhyay, L. Higham, and K. Seyffarth. Dynamic and self-stabilizing distributed matching. In *Proceedings of the Twenty-First Annual Symposium on Principles of Distributed Computing*, pages 290–297. ACM Press, 2002.

[2] E. W. Dijkstra. Self-stabilizing systems in spite of distributed control. *Comm. ACM*, 17(11):643–644, Jan. 1974.

[3] E. W. Dijkstra. A belated proof of self-stabilization. *Distributed Computing*, 1(1):5–6, 1986.

[4] S. Dolev. *Self-Stabilization.* MIT Press, 2000.

[5] W. Goddard, S. T. Hedetniemi, D. P. Jacobs, and P. K. Srimani. Fault tolerant algorithms for orderings and colorings. In *Proceedings of the IPDPS Workshop on Advances in Parallel and Distributed Computational Models (APDCM04)*, 2004.

[6] S. M. Hedetniemi, S. T. Hedetniemi, D. P. Jacobs, and P. K. Srimani. Self-stabilizing algorithms for minimal dominating sets and maximal independent sets. *Comput. Math. Appl.*, 46(5-6):805–811, 2003.

[7] S.-C. Hsu and S.-T. Huang. A self-stabilizing algorithm for maximal matching. *Inform. Process. Lett.*, 43:77–81, 1992.

[8] Z. Shi, W. Goddard, and S. T. Hedetniemi. An anonymous self-stabilizing algorithm for 1-maximal independent sets in trees. *Inform. Process. Lett.*, 91:77–83, 2004.

[9] S. Shukla, D. Rosenkrantz, and S. Ravi. Observations on self-stabilizing graph algorithms for anonymous networks. In *Proceedings of the Second Workshop on Self-Stabilizing Systems*, page 7.17.15, 1995.

[10] G. Tel. *Introduction to Distributed Algorithms, Second Edition.* Cambridge University Press, Cambridge UK, 2000.